

# TCP-Chat-Server in C++

## Codedokumentation für die Fachprüfung

Christine Münzberg

28. Juli 2026

### Inhaltsverzeichnis

|          |                                            |          |
|----------|--------------------------------------------|----------|
| <b>1</b> | <b>Projektübersicht</b>                    | <b>2</b> |
| <b>2</b> | <b>Architektur und Ablauf</b>              | <b>2</b> |
| 2.1      | Wichtige Konzepte . . . . .                | 2        |
| <b>3</b> | <b>Bauanleitung</b>                        | <b>2</b> |
| 3.1      | Variante A: direkt mit g++ . . . . .       | 2        |
| 3.2      | Variante B: über CMake . . . . .           | 3        |
| <b>4</b> | <b>Quellcode: server.cpp</b>               | <b>3</b> |
| <b>5</b> | <b>Quellcode: client.cpp</b>               | <b>5</b> |
| <b>6</b> | <b>Build-Konfiguration: CMakeLists.txt</b> | <b>7</b> |
| <b>7</b> | <b>Fehlerbehandlung im Code</b>            | <b>8</b> |
| <b>8</b> | <b>Mögliche Erweiterungen</b>              | <b>8</b> |

# 1 Projektübersicht

Dieses Projekt implementiert einen einfachen 1:1-Chat über TCP-Sockets unter Windows (Winsock2-API). Das Projekt besteht aus zwei eigenständigen Programmen:

- **server.cpp** – wartet auf eine eingehende Verbindung und tauscht anschließend abwechselnd Nachrichten mit dem Client aus.
- **client.cpp** – baut eine Verbindung zum Server auf (Standard: 127.0.0.1:9000) und kommuniziert im gleichen Frage-Antwort-Rhythmus.

Beide Programme werden unabhängig voneinander kompiliert und als separate ausführbare Dateien (**server.exe**, **client.exe**) gestartet. Der Build erfolgt entweder direkt mit **g++** oder über **CMakeLists.txt**.

## 2 Architektur und Ablauf

Beide Seiten folgen dem klassischen Winsock-Lebenszyklus. Der Unterschied liegt darin, dass der Server passiv auf eine Verbindung wartet (**bind** → **listen** → **accept**), während der Client aktiv eine Verbindung aufbaut (**connect**).

| Server                   | Client                   |
|--------------------------|--------------------------|
| WSAStartup               | WSAStartup               |
| socket                   | socket                   |
| bind                     | –                        |
| listen                   | –                        |
| accept                   | connect                  |
| recv / send (Schleife)   | send / recv (Schleife)   |
| closesocket / WSACleanup | closesocket / WSACleanup |

Nach dem Verbindungsaufbau laufen Server und Client in einer Schleife, in der abwechselnd eine Nachricht empfangen und eine Nachricht gesendet wird. Die Eingabe **"exit"** auf einer der beiden Seiten beendet den Chat kontrolliert auf beiden Seiten.

### 2.1 Wichtige Konzepte

**TCP vs. UDP** TCP ist verbindungsorientiert und garantiert zuverlässige, geordnete Zustellung – passend für einen Chat, bei dem keine Nachricht verloren gehen soll. UDP wäre verbindungslos und ungeordnet, aber schneller.

**Blockierende Aufrufe** **accept()** und **recv()** blockieren das Programm, bis ein Ereignis eintritt (neue Verbindung bzw. neue Daten). Dadurch läuft der Code synchron ohne aktives Polling.

**Zweck des Puffers** **char puffer[PUFFER]** nimmt die empfangenen Rohdaten (Bytes) auf. Die Größe (1024 Byte) begrenzt die maximale Nachrichtenlänge pro **recv()**-Aufruf.

**Umwandlung **char\*** → **std::string**** Nach dem Empfang wird der Puffer an der empfangenen Byte-Anzahl mit **'\0'** terminiert und anschließend in ein **std::string** überführt, um komfortabel mit C++-Stringfunktionen weiterzuarbeiten.

## 3 Bauanleitung

### 3.1 Variante A: direkt mit g++

```
g++ -o server.exe server.cpp -lws2_32 -std=c++17
g++ -o client.exe client.cpp -lws2_32 -std=c++17
```

## 3.2 Variante B: über CMake

```
mkdir build
cd build
cmake ..
cmake --build .
```

Der Linker-Flag `-lws2_32` bindet die Windows-Socket-Bibliothek ein und ist zwingend nötig, da Winsock2 (im Gegensatz zu den POSIX-Sockets unter Linux) eine separate Bibliothek ist, die nicht automatisch verlinkt wird.

## 4 Quellcode: server.cpp

Der Server initialisiert Winsock, erstellt einen Socket, bindet ihn an Port 9000, wartet auf eine Verbindung und tauscht danach Nachrichten mit dem Client aus, bis eine Seite `"exit"` sendet.

```
1 // =====
2 // server.cpp - Einfacher TCP-Chat-Server für Windows
3 // Fachprüfungsprojekt: C++ Netzwerkprogrammierung
4 // =====
5
6 #include <iostream>          // Für cout und cin
7 #include <string>            // Für std::string
8 #include <winsock2.h>        // Windows-Socket-Bibliothek
9
10 // Winsock2-Bibliothek beim Linker einbinden
11 #pragma comment(lib, "ws2_32.lib")
12
13 // Konstanten
14 const int PORT = 9000;      // Port, auf dem der Server lauscht
15 const int PUFFER = 1024;    // Maximale Nachrichtenlänge in Bytes
16
17 int main() {
18
19     // -----
20     // SCHRITT 1: Winsock initialisieren
21     // WSASStartup teilt Windows mit, dass wir Sockets verwenden.
22     // MAKEWORD(2,2) gibt an, dass wir Version 2.2 nutzen wollen.
23     // -----
24     WSADATA wsaData;
25     if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
26         std::cerr << "Fehler: WSAStartup fehlgeschlagen.\n";
27         return 1;
28     }
29     std::cout << "=== Chat-Server gestartet ===\n";
30
31     // -----
32     // SCHRITT 2: Server-Socket erstellen
33     // AF_INET = IPv4-Adressfamilie
34     // SOCK_STREAM = TCP (verbindungsorientiert, zuverlässig)
35     // IPPROTO_TCP = TCP-Protokoll
36     // -----
37     SOCKET serverSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
38     if (serverSocket == INVALID_SOCKET) {
39         std::cerr << "Fehler: Socket-Erstellung fehlgeschlagen.\n";
40         WSACleanup();
```

```
41     return 1;
42 }
43 std::cout << "Socket erfolgreich erstellt.\n";
44
45 // -----
46 // SCHRITT 3: Socket-Adresse konfigurieren
47 // sockaddr_in enthält IP-Adresse und Port.
48 // INADDR_ANY bedeutet: auf allen verfügbaren Netzwerkkarten lauschen.
49 // htons() konvertiert den Port in Netzwerk-Byte-Reihenfolge.
50 // -----
51 sockaddr_in serverAddr;
52 serverAddr.sin_family      = AF_INET;
53 serverAddr.sin_addr.s_addr = INADDR_ANY;
54 serverAddr.sin_port        = htons(PORT);
55
56 // -----
57 // SCHRITT 4: Socket an Port binden (bind)
58 // Verknüpft den Socket mit der Adresse und dem Port.
59 // -----
60 if (bind(serverSocket, (sockaddr*)&serverAddr, sizeof(serverAddr)) ==
SOCKET_ERROR) {
61     std::cerr << "Fehler: Bind fehlgeschlagen.\n";
62     closesocket(serverSocket);
63     WSACleanup();
64     return 1;
65 }
66 std::cout << "Socket an Port " << PORT << " gebunden.\n";
67
68 // -----
69 // SCHRITT 5: Auf Verbindungen lauschen (listen)
70 // Der zweite Parameter (1) ist die Warteschlangenlänge
71 // (wie viele Verbindungsanfragen zwischengespeichert werden).
72 // -----
73 if (listen(serverSocket, 1) == SOCKET_ERROR) {
74     std::cerr << "Fehler: Listen fehlgeschlagen.\n";
75     closesocket(serverSocket);
76     WSACleanup();
77     return 1;
78 }
79 std::cout << "Warte auf Client-Verbindung auf Port " << PORT << "... \n";
80
81 // -----
82 // SCHRITT 6: Client-Verbindung annehmen (accept)
83 // accept() blockiert das Programm, bis ein Client sich verbindet.
84 // Gibt einen neuen Socket zurück, der für die Kommunikation genutzt wird.
85 // -----
86 SOCKET clientSocket = accept(serverSocket, nullptr, nullptr);
87 if (clientSocket == INVALID_SOCKET) {
88     std::cerr << "Fehler: Accept fehlgeschlagen.\n";
89     closesocket(serverSocket);
90     WSACleanup();
91     return 1;
92 }
93 std::cout << "Client verbunden! Tippe Nachrichten oder 'exit' zum Beenden.\n\n";
94
95 // -----
96 // SCHRITT 7: Kommunikations-Loop
97 // Abwechselnd: Nachricht vom Client empfangen, dann senden.
98 // -----
99 char puffer[PUFFER]; // Empfangspuffer
100
101 while (true) {
102
```

```

103 // --- Nachricht vom Client empfangen (recv) ---
104 // recv() blockiert, bis Daten ankommen.
105 // Gibt die Anzahl empfangener Bytes zurück (0 = Verbindung getrennt).
106 int empfangen = recv(clientSocket, puffer, PUFFER - 1, 0);
107 if (empfangen <= 0) {
108     std::cout << "\nClient hat die Verbindung getrennt.\n";
109     break;
110 }
111 puffer[empfangen] = '\0'; // Null-Terminierung für std::string
112 std::string nachrichtVomClient(puffer);
113
114 // Prüfen ob Client "exit" gesendet hat
115 if (nachrichtVomClient == "exit") {
116     std::cout << "Client hat den Chat beendet.\n";
117     break;
118 }
119 std::cout << "[Client]: " << nachrichtVomClient << "\n";
120
121 // --- Nachricht vom Server-Nutzer eingeben und senden (send) ---
122 std::cout << "[Server]: ";
123 std::string nachrichtAnClient;
124 std::getline(std::cin, nachrichtAnClient);
125
126 // Prüfen ob Server "exit" eingegeben hat
127 if (nachrichtAnClient == "exit") {
128     send(clientSocket, "exit", 4, 0);
129     std::cout << "Chat vom Server beendet.\n";
130     break;
131 }
132
133 // Nachricht an den Client senden
134 int gesendet = send(clientSocket, nachrichtAnClient.c_str(),
135                     (int)nachrichtAnClient.size(), 0);
136 if (gesendet == SOCKET_ERROR) {
137     std::cerr << "Fehler beim Senden der Nachricht.\n";
138     break;
139 }
140 }
141
142 // -----
143 // SCHRITT 8: Aufräumen - Sockets schließen & Winsock beenden
144 // Immer in umgekehrter Reihenfolge: zuerst Client-, dann Server-Socket.
145 // -----
146 closesocket(clientSocket);
147 closesocket(serverSocket);
148 WSACleanup();
149
150 std::cout << "Server wurde sauber beendet.\n";
151 return 0;
152 }

```

Listing 1: server.cpp – vollständiger Quellcode

## 5 Quellcode: client.cpp

Der Client baut eine Verbindung zum Server auf (Loopback-Adresse 127.0.0.1, Port 9000) und sendet als Erster eine Nachricht, bevor er auf die Antwort des Servers wartet.

```

1 // =====
2 // client.cpp - Einfacher TCP-Chat-Client für Windows
3 // Fachprüfungsprojekt: C++ Netzwerkprogrammierung
4 // =====

```

```

5
6 #include <iostream>           // Für cout und cin
7 #include <string>             // Für std::string
8 #include <winsock2.h>         // Windows-Socket-Bibliothek
9 #include <ws2tcpip.h>         // Für inet_pton (IP-Adresse konvertieren)
10
11 // Winsock2-Bibliothek beim Linker einbinden
12 #pragma comment(lib, "ws2_32.lib")
13
14 // Konstanten
15 const char* SERVER_IP = "127.0.0.1"; // Loopback (Server läuft lokal)
16 const int PORT = 9000;               // Port des Servers
17 const int PUFFER = 1024;             // Maximale Nachrichtenlänge in Bytes
18
19 int main() {
20
21     // -----
22     // SCHRITT 1: Winsock initialisieren (gleich wie beim Server)
23     // -----
24     WSADATA wsaData;
25     if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
26         std::cerr << "Fehler: WSAStartup fehlgeschlagen.\n";
27         return 1;
28     }
29     std::cout << "=== Chat-Client gestartet ===\n";
30
31     // -----
32     // SCHRITT 2: Client-Socket erstellen
33     // Gleiche Parameter wie beim Server-Socket.
34     // -----
35     SOCKET clientSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
36     if (clientSocket == INVALID_SOCKET) {
37         std::cerr << "Fehler: Socket-Erstellung fehlgeschlagen.\n";
38         WSACleanup();
39         return 1;
40     }
41     std::cout << "Socket erstellt. Verbinde mit Server...\n";
42
43     // -----
44     // SCHRITT 3: Server-Adresse konfigurieren
45     // inet_pton() konvertiert die IP-Adresse (String) in Binärformat.
46     // -----
47     sockaddr_in serverAddr;
48     serverAddr.sin_family = AF_INET;
49     serverAddr.sin_port = htons(PORT);
50     inet_pton(AF_INET, SERVER_IP, &serverAddr.sin_addr);
51
52     // -----
53     // SCHRITT 4: Verbindung zum Server aufbauen (connect)
54     // connect() sendet einen TCP-Verbindungsaufbau (SYN-Paket).
55     // Blockiert bis die Verbindung steht oder ein Fehler auftritt.
56     // -----
57     if (connect(clientSocket, (sockaddr*)&serverAddr, sizeof(serverAddr)) ==
        SOCKET_ERROR) {
58         std::cerr << "Fehler: Verbindung zum Server fehlgeschlagen.\n";
59         std::cerr << "Ist der Server gestartet und lauscht auf Port " << PORT << "?\n";
60         closesocket(clientSocket);
61         WSACleanup();
62         return 1;
63     }
64     std::cout << "Verbunden mit Server " << SERVER_IP << ":" << PORT << "\n";
65     std::cout << "Tippe Nachrichten oder 'exit' zum Beenden.\n\n";

```

```

66
67 // -----
68 // SCHRITT 5: Kommunikations-Loop
69 // Abwechselnd: Nachricht eingeben & senden, dann Antwort empfangen.
70 // -----
71 char puffer[PUFFER]; // Empfangspuffer
72
73 while (true) {
74
75     // --- Nachricht vom Client-Nutzer eingeben ---
76     std::cout << "[Du]: ";
77     std::string nachricht;
78     std::getline(std::cin, nachricht);
79
80     // Prüfen ob Nutzer "exit" eingegeben hat
81     if (nachricht == "exit") {
82         send(clientSocket, "exit", 4, 0);
83         std::cout << "Chat beendet.\n";
84         break;
85     }
86
87     // --- Nachricht an Server senden (send) ---
88     int gesendet = send(clientSocket, nachricht.c_str(),
89                         (int)nachricht.size(), 0);
90     if (gesendet == SOCKET_ERROR) {
91         std::cerr << "Fehler beim Senden.\n";
92         break;
93     }
94
95     // --- Antwort vom Server empfangen (recv) ---
96     int empfangen = recv(clientSocket, puffer, PUFFER - 1, 0);
97     if (empfangen <= 0) {
98         std::cout << "Server hat die Verbindung getrennt.\n";
99         break;
100    }
101    puffer[empfangen] = '\0';
102    std::string antwort(puffer);
103
104    // Prüfen ob Server "exit" gesendet hat
105    if (antwort == "exit") {
106        std::cout << "Server hat den Chat beendet.\n";
107        break;
108    }
109    std::cout << "[Server]: " << antwort << "\n";
110 }
111
112 // -----
113 // SCHRITT 6: Aufräumen
114 // -----
115 closesocket(clientSocket);
116 WSACleanup();
117
118 std::cout << "Client wurde sauber beendet.\n";
119 return 0;
120 }

```

Listing 2: client.cpp – vollständiger Quellcode

## 6 Build-Konfiguration: CMakeLists.txt

Definiert zwei separate Targets (`server`, `client`) und verlinkt unter Windows jeweils die Winsock2-Bibliothek.

```
1 cmake_minimum_required(VERSION 3.10)
2 project(ChatProjekt)
3
4 set(CMAKE_CXX_STANDARD 17)
5
6 # Server-Executable erstellen
7 add_executable(server server.cpp)
8
9 # Client-Executable erstellen
10 add_executable(client client.cpp)
11
12 # Winsock2-Bibliothek für beide Targets einbinden (nur Windows nötig)
13 if(WIN32)
14     target_link_libraries(server ws2_32)
15     target_link_libraries(client ws2_32)
16 endif()
```

Listing 3: CMakeLists.txt

## 7 Fehlerbehandlung im Code

Beide Programme prüfen nach jedem kritischen Winsock-Aufruf (`WSAStartup`, `socket`, `bind`, `listen`, `accept`, `connect`, `send`) den Rückgabewert und brechen bei Fehlern mit einer Meldung auf `std::cerr` sowie kontrolliertem Aufräumen (`closesocket`, `WSACleanup`) ab.

| Fehlermeldung                         | Ursache                         | Lösung                                      |
|---------------------------------------|---------------------------------|---------------------------------------------|
| 'g++' is not recognized               | PATH nicht korrekt gesetzt      | Neues Terminal öffnen, PATH prüfen          |
| undefined reference to WSAStartup     | -lws2_32 fehlt beim Kompilieren | Linker-Flag ergänzen                        |
| winsock2.h: No such file or directory | Falscher/kein Windows-Compiler  | MinGW-w64-Toolchain installieren            |
| Bind fehlgeschlagen                   | Port 9000 bereits belegt        | Alten Serverprozess beenden, erneut starten |

## 8 Mögliche Erweiterungen

- Mehrere Clients gleichzeitig (ein Thread pro Verbindung)
- Zeitstempel und Absendername je Nachricht
- Broadcast an alle verbundenen Clients statt 1:1
- Robustere Fehlerbehandlung, z. B. Timeout bei `recv`